

# Retention e contenimento della tabella di Audit su SQL Server Express

Compressione PAGE, retention temporale automatica (36 mesi) e sentinella sui 10 GB del database. Cosa è stato implementato, perché, e dove leggerlo nel codice.

Data: 15 giugno 2026 · Migration di riferimento: `024_AuditCompressionRetention.sql` · Origine: `docs/audit-dimensionamento-sql-express.pdf`

## 1. Contesto e decisione

La nota di dimensionamento `audit-dimensionamento-sql-express.pdf` evidenzia che il vincolo reale di SQL Server Express non è il numero di righe ma il **tetto di 10 GB per database**, condiviso con tutti i dati. La tabella `fatt.Audit` registra il dettaglio JSON delle scritture (snapshot/diff): è il payload a dominare la riga.

Poiché il progetto adotta già il **modify-delta** (`AuditDettaglio.Diff` salva solo i campi cambiati), siamo già nello scenario «leggero». Si è scelto comunque di mettere in atto la raccomandazione della nota («definire da subito una retention»), con una precisazione di metodo:

**Per una tabella di audit la soglia corretta è l'età dei dati, non la loro mole.** Cancellare in base alla dimensione produrrebbe una perdita di storia silenziosa e imprevedibile durante i picchi. La dimensione è invece il criterio giusto per una *sentinella che avvisa* sull'intero database, non per cancellare l'audit.

**Decisione (concordata):** retention temporale `automatica + manuale`, finestra `36 mesi`, sentinella di allarme a `8 GB`, su fondamenta di compressione PAGE.

## 2. Strategia a tre livelli

| Livello                  | Cosa fa                                                                                                               | Natura                                           |
|--------------------------|-----------------------------------------------------------------------------------------------------------------------|--------------------------------------------------|
| 1. Compressione PAGE     | Comprime dati e indici di <code>fatt.Audit</code> e <code>fatt.Log</code> (testo ripetitivo → ~50/70%).               | Schema (migration 024), una tantum, trasparente. |
| 2. Retention temporale   | Elimina le righe di audit più vecchie di N mesi. <b>L'unico passo che cancella, e solo per età.</b>                   | Manuale (pulsante) + automatica (job).           |
| 3. Sentinella dimensione | Se i file dati superano la soglia, scrive un Warning in <code>fatt.Log</code> . <b>Avvisa soltanto, non cancella.</b> | Automatica (nel job).                            |

## 3. Livello 1 — Compressione PAGE (migration 024)

`src/ICMFatturazioni.Web/Migrations/024_AuditCompressionRetention.sql`

Una proprietà permanente di tabella: il `REBUILD` comprime subito i dati esistenti, poi ogni `INSERT / UPDATE` è compresso automaticamente da SQL Server, senza alcuna modifica al codice C# (la lettura decomprime al volo). `ALTER INDEX ALL` copre l'indice clustered (i dati) e tutti i non-clustered. Idempotente: ricostruisce solo se non è già in PAGE.

```
-- fatt.Audit - comprime dati (clustered) + tutti gli indici non-clustered
IF EXISTS (
  SELECT 1 FROM sys.partitions p
    JOIN sys.tables t ON p.object_id = t.object_id
    JOIN sys.schemas s ON t.schema_id = s.schema_id
 WHERE s.name = N'fatt' AND t.name = N'Audit'
    AND p.data_compression_desc <> N'PAGE')
BEGIN
  ALTER INDEX ALL ON fatt.Audit REBUILD WITH (DATA_COMPRESSION = PAGE);
END
```

**Verificato a DB:** compressione `PAGE` attiva su tutti gli 8 indici di `fatt.Audit` (4) e `fatt.Log` (4), dati + indici.

## 4. Livello 2 — Retention temporale

## 4.1 Repository — il DELETE per soglia

src/ICMFatturazioni.Web/Repositories/AuditRepository.cs · interfaccia IAuditRepository.cs

```
public async Task<int> PurgaPrecedentiAsync(DateTime sogliaUtc, CancellationToken ct = default)
{
    using var conn = await _connectionFactory.CreateOpenConnectionAsync(ct);
    return await conn.ExecuteAsync(new CommandDefinition(
        "DELETE FROM fatt.Audit WHERE TimestampUtc < @SogliaUtc;",
        new { SogliaUtc = sogliaUtc }, cancellationToken: ct));
}
```

## 4.2 Manager — da «mesi» a soglia (testabile)

src/ICMFatturazioni.Web/Managers/AuditManager.cs · interfaccia IAuditManager.cs

La soglia è calcolata col `TimeProvider` iniettato (pilottabile nei test). `Math.Abs` difensivo: un input negativo accidentale non purga il futuro. La purga di per sé **non** viene a sua volta auditata (è manutenzione, non un'operazione di dominio dell'utente).

```
public Task<int> PurgaPrecedentiAsync(int mesi, CancellationToken ct = default)
{
    // Soglia col TimeProvider (testabile); |mesi| difensivo.
    var soglia = _clock.GetUtcNow().UtcDateTime.AddMonths(-Math.Abs(mesi));
    return _repository.PurgaPrecedentiAsync(soglia, ct);
}
```

# 5. Livello 2+3 — Job automatico e sentinella

## 5.1 Logica di manutenzione (testabile, fuori dal BackgroundService)

src/ICMFatturazioni.Web/Manutenzione/AuditManutenzione.cs

Due responsabilità distinte in un ciclo: **(1)** retention; **(2)** sentinella. Il Warning è emesso con `ILogger`: il `DbLoggerProvider` già esistente lo **persiste in** `fatt.Log`, dove l'amministratore lo vede in `/admin/log`. Riutilizza l'infrastruttura di logging senza nuovi canali.

```
// 1. Retention temporale (idempotente).
var eliminate = await _auditManager.PurgaPrecedentiAsync(_options.MesiConservazione, ct);

// 2. Sentinella sulla dimensione dei file dati.
var dimensioneMb = await _sizeRepository.GetDimensioneDatiMbAsync(ct);
var allarme = dimensioneMb >= _options.SogliaAllarmeGb * 1024;
if (allarme)
    _logger.LogWarning("Database a {Mb} MB, oltre la soglia di {Gb} GB ...",
        dimensioneMb, _options.SogliaAllarmeGb);
```

La dimensione è letta dai soli file dati (esclude il log di transazione, che non concorre al tetto di Express):

src/ICMFatturazioni.Web/Repositories/DatabaseSizeRepository.cs

```
SELECT CAST(ISNULL(SUM(CAST(size AS BIGINT)), 0) * 8 / 1024 AS INT)
FROM sys.database_files WHERE type = 0; -- 0 = ROWS (dati); 1 = LOG (escluso)
```

## 5.2 Il BackgroundService schedulato

src/ICMFatturazioni.Web/Manutenzione/AuditRetentionService.cs

Gira a cadenza configurata (default 24 h), dopo un ritardo iniziale di 2 minuti per non competere con l'avvio. Crea uno *scope* per ciclo (i Manager/Repository sono Scoped) e **cattura e logga i propri errori** via `ILogManager` (Regola 6: un'eccezione non gestita in un BackgroundService sarebbe invisibile). Disabilitabile via `AuditRetention:JobAbilitato`.

**Manuale e automatico non si scontrano.** La purga «elimina i più vecchi di 36 mesi» è idempotente: eseguita due volte di fila, la seconda non trova nulla. Il job la applica da solo; il pulsante è un «fallo adesso» a mano.

# 6. Pulsante manuale in /admin/audit

src/ICMFatturazioni.Web/Components/Pages/Admin/AuditPage.razor

Mirror del pulsante di purga già presente in `LogPage`. L'etichetta e la finestra derivano dalla configurazione (`IOptions<AuditRetentionOptions>`); riusa `ConfermaEliminazioneDialog` per la conferma e mostra l'esito in uno snackbar.

```
<MudButton Color="Color.Error" StartIcon="@Icons.Material.Filled.DeleteSweep"
  OnClick="PurgaAsync">Purga &gt; @MesiConservazione mesi</MudButton>

// ...nel code-behind, dopo conferma:
var eliminate = await AuditManager.PurgaPrecedentiAsync(mesi);
Snackbar.Add($"Eliminate {eliminate} righe di audit.", Severity.Success);
await _grid.ReloadServerData();
```

## 7. Configurazione

`src/ICMFatturazioni.Web/Manutenzione/AuditRetentionOptions.cs` · `appsettings.json` (sezione `AuditRetention`)

| Parametro         | Default | Significato                                                                                  |
|-------------------|---------|----------------------------------------------------------------------------------------------|
| JobAbilitato      | true    | Se <code>false</code> , resta solo la purga manuale (niente job automatico).                 |
| MesiConservazione | 36      | Finestra di retention. Generosa: con compressione + modify-delta lo spazio non è il vincolo. |
| IntervalloOreJob  | 24      | Cadenza del job. La retention è idempotente: non serve girare più spesso.                    |
| SogliaAllarmeGb   | 8       | 80% del tetto di 10 GB. Superata → Warning in <code>fatt.Log</code> (non cancella).          |

**Nota di conservazione (decisione di business).** Trattandosi di un gestionale di fatturazione, in Italia le scritture contabili si conservano 10 anni (art. 2220 c.c.). L'audit non è il documento fiscale ma il log delle modifiche: 36 mesi è un default prudente e **configurabile**. Per allinearsi alla conservazione decennale, impostare `MesiConservazione = 120`.

## 8. Test e verifica

### 8.1 Test automatici (xUnit) — +5

| Test                                                    | Verifica                                                                                                |
|---------------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| <code>AuditManagerTests.PurgaPrecedentiAsync_...</code> | La purga elimina solo le righe oltre la finestra (mesi → soglia).                                       |
| <code>AuditManutenzioneTests</code> (×4)                | Retention elimina solo l'oltre-finestra; sentinella tace sotto soglia, avvisa oltre e al valore esatto. |

Fake aggiunti: `FakeDatabaseSizeRepository`, `CapturingLogger<T>`, `TestTimeProvider`; `FakeAuditRepository` / `FakeAuditManager` estesi con la purga.

**Build: 0 warning / 0 errori** · **Test: 262 verdi** (erano 257). · **Migration 024** applicata al DB di sviluppo.

### 8.2 Smoke test end-to-end (Playwright, su `/admin/audit`)

| Scenario         | Esito                                                                                                                         |
|------------------|-------------------------------------------------------------------------------------------------------------------------------|
| Job automatico   | <b>PASS</b> — una riga retrodatata (−40 mesi) è stata eliminata <b>dal job da solo</b> al primo ciclo.                        |
| Pulsante manuale | <b>PASS</b> — dialog «Operazioni più vecchie di 36 mesi» → « <b>Eliminate 1 righe di audit.</b> »; la riga recente è rimasta. |
| Sentinella       | <b>PASS</b> — 0 Warning in <code>fatt.Log</code> (DB piccolo, sotto soglia: comportamento atteso).                            |

## 9. File toccati

| File                                                      | Tipo  |
|-----------------------------------------------------------|-------|
| <code>Migrations/024_AuditCompressionRetention.sql</code> | nuovo |
| <code>Manutenzione/AuditRetentionOptions.cs</code>        | nuovo |

|                                                                          |             |
|--------------------------------------------------------------------------|-------------|
| Manutenzione/IAuditManutenzione.cs · AuditManutenzione.cs                | nuovo       |
| Manutenzione/AuditRetentionService.cs                                    | nuovo       |
| Repositories/IDatabaseSizeRepository.cs · DatabaseSizeRepository.cs      | nuovo       |
| Repositories/AuditRepository.cs · Interfaces/IAuditRepository.cs         | + purga     |
| Managers/AuditManager.cs · Interfaces/IAuditManager.cs                   | + purga     |
| Components/Pages/Admin/AuditPage.razor                                   | + pulsante  |
| Program.cs · appsettings.json                                            | DI + config |
| tests/.../AuditManagerTests.cs · AuditManutenzioneTests.cs · fake/helper | + test      |

ICMFatturazioni — Nota tecnica di implementazione «Retention e contenimento dell'Audit». Generata il 15 giugno 2026. Le scelte rispettano le convenzioni del progetto (architettura layered UI→Manager→Repository, Regola 6 logging, Regola 7 audit, mirror ICMVerbalì) descritte in [CLAUDE.md](#).